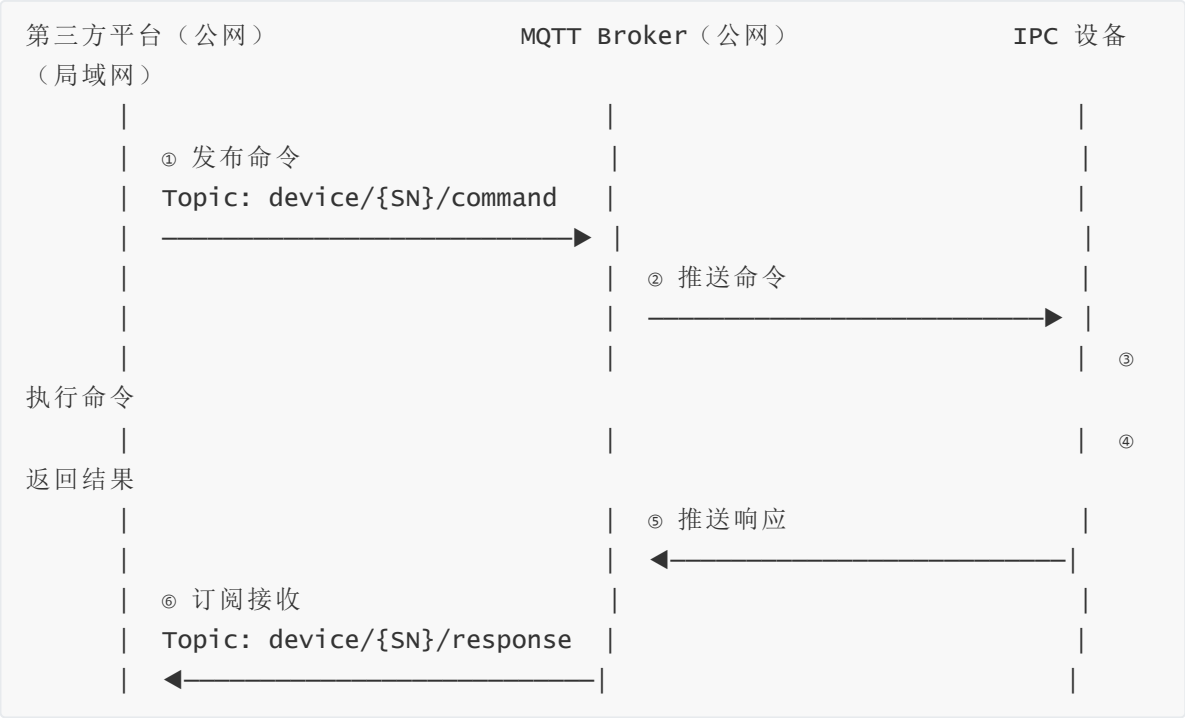


IPC 摄像头 MQTT-SDK 第三方平台对接使用手册

1. 概述

本文档所有示例均以 C++ 编写，基于 **Eclipse Paho MQTT C++** 客户端库（`paho-mqttpp3`），展示完整的连接、订阅、命令发送与响应接收流程。

通信架构



- 设备在局域网内，作为 MQTT 客户端主动连接公网 Broker，无需公网 IP
- 第三方平台连接同一个 Broker，通过 Topic 规则与设备双向通信
- 鉴权由 MQTT Broker 连接层保障，消息层不做额外 Token 校验

2. 环境准备

2.1 依赖库

库	说明
<code>paho-mqttpp3</code>	Eclipse Paho MQTT C++ 客户端库
<code>paho-mqtt3as</code>	Paho MQTT C 异步+SSL 库（ <code>paho-mqttpp3</code> 的底层依赖）
<code>nlohmann/json</code>	JSON 解析库（或使用其他 JSON 库均可）

2.2 编译安装 (Linux)

```
# 安装 Paho MQTT C 库
git clone https://github.com/eclipse/paho.mqtt.c.git
cd paho.mqtt.c
cmake -B build -DPAHO_WITH_SSL=ON
cmake --build build --target install

# 安装 Paho MQTT C++ 库
git clone https://github.com/eclipse/paho.mqtt.cpp.git
cd paho.mqtt.cpp
cmake -B build -DPAHO_BUILD_SHARED=ON -DPAHO_WITH_MQTT_C=ON
cmake --build build --target install
```

3. MQTT Broker 连接信息

参数	值
Broker 地址	183.129.224.253
Broker 端口	1883
协议	MQTT 3.1.1 / MQTT 5.0
连接认证	平台账号密码
心跳间隔	建议 60 秒
Clean Session	建议 true

4. Topic 规则

Topic	方向	QoS	说明
device/{SN}/command	平台 → 设备	1	平台向设备下发命令
device/{SN}/response	设备 → 平台	1	设备返回命令执行结果
device/{SN}/event	设备 → 平台	0	设备主动上报事件/告警

- {SN} 为设备序列号，例如 IPC20260001
- 平台需要订阅 device/{SN}/response 和 device/{SN}/event
- 平台需要发布到 device/{SN}/command

5. 消息格式

5.1 命令请求（平台 → 设备）

```
{
  "Command": "NET_TV_命令名",
  "RequestId": "请求唯一标识（可选）",
  "Data": {}
}
```

字段	类型	必填	说明
Command	string	是	SDK 命令名
RequestId	string	否	请求标识，响应中原样返回，用于关联请求和响应
Data	object	否	业务数据，缺失时默认 {}

5.2 命令响应（设备 → 平台）

```
{
  "Command": "NET_TV_命令名",
  "RequestId": "请求唯一标识（原样返回）",
  "Return": 0,
  "Data": {}
}
```

字段	类型	说明
Command	string	与请求一致
RequestId	string	与请求一致
Return	int	返回码，见错误码表
Data	object	响应数据

6. 错误码

Return	说明
0	成功
-1	系统未就绪（MQTT 未连接或平台未登录）
-2	不支持的命令（命令名未注册）
其他负值	命令执行失败（设备内部错误）

7. 平台端通用封装

以下为第三方平台端的通用 MQTT 连接、命令发送与响应接收封装，后续所有命令示例均基于此封装。

7.1 头文件

```
/**
 * @brief IPC 设备 MQTT 客户端封装
 */
#pragma once

#include <mqtt/async_client.h>
#include <nlohmann/json.hpp>
#include <string>
#include <functional>
#include <mutex>
#include <condition_variable>
#include <map>

using json = nlohmann::json;

class IpcMqttClient
{
public:
    /* 响应回调类型: RequestId -> 响应 JSON */
    using ResponseCallback = std::function<void(const std::string
    &strRequestId, const json &jResponse)>;

    /* 事件回调类型: 事件 JSON */
    using EventCallback = std::function<void(const json &jEvent)>;

    IpcMqttClient(const std::string &strBroker,
                  int nPort,
                  const std::string &strUsername,
                  const std::string &strPassword,
                  const std::string &strDeviceSn);
    ~IpcMqttClient();

    /* 连接 Broker, 订阅响应和事件 Topic */
    int connect();

    /* 断开连接 */
    void disconnect();

    /* 发送命令并同步等待响应（超时毫秒） */
    json sendCommand(const std::string &strCommand,
                    const json &jData = json::object(),
                    int nTimeoutMs = 10000);

    /* 设置异步响应回调 */
    void setResponseCallback(ResponseCallback cb);
```

```

    /* 设置事件回调 */
    void setEventCallback(EventCallback cb);

private:
    void onMessage(const std::string &strTopic, const std::string
&strPayload);
    std::string generateRequestId();

    std::string m_strBroker;
    int m_nPort;
    std::string m_strUsername;
    std::string m_strPassword;
    std::string m_strDeviceSn;
    std::string m_strCommandTopic;
    std::string m_strResponseTopic;
    std::string m_strEventTopic;

    mqtt::async_client *m_pClient = nullptr;
    mqtt::connect_options m_connOpts;

    ResponseCallback m_fnResponseCallback;
    EventCallback m_fnEventCallback;

    /* 同步等待响应 */
    std::mutex m_mtxResponse;
    std::condition_variable m_cvResponse;
    std::map<std::string, json> m_mapPendingResponses;
};

```

7.2 实现文件

```

#include "IpcMqttClient.h"
#include <chrono>
#include <random>
#include <sstream>
#include <iomanip>

IpcMqttClient::IpcMqttClient(const std::string &strBroker,
                             int nPort,
                             const std::string &strUsername,
                             const std::string &strPassword,
                             const std::string &strDeviceSn)
    : m_strBroker(strBroker)
    , m_nPort(nPort)
    , m_strUsername(strUsername)
    , m_strPassword(strPassword)
    , m_strDeviceSn(strDeviceSn)
{
    std::string strAddr = "tcp://" + m_strBroker + ":" +
std::to_string(m_nPort);
    m_strCommandTopic = "device/" + m_strDeviceSn + "/command";
}

```

```

m_strResponseTopic = "device/" + m_strDeviceSn + "/response";
m_strEventTopic    = "device/" + m_strDeviceSn + "/event";

m_pClient = new mqtt::async_client(strAddr, "platform_" +
m_strDeviceSn);

m_connOpts.set_user_name(m_strUsername);
m_connOpts.set_password(m_strPassword);
m_connOpts.set_keep_alive_interval(60);
m_connOpts.set_clean_session(true);

// 设置消息回调
m_pClient->set_message_callback([this](mqtt::const_message_ptr msg) {
    onMessage(msg->get_topic(), msg->to_string());
});
}

IpcMqttClient::~IpcMqttClient()
{
    disconnect();
    delete m_pClient;
}

int IpcMqttClient::connect()
{
    try {
        auto connRsp = m_pClient->connect(m_connOpts)->get();
        // 订阅响应和事件 Topic
        m_pClient->subscribe(m_strResponseTopic, 1);
        m_pClient->subscribe(m_strEventTopic, 0);
        return 0;
    } catch (const mqtt::exception &e) {
        std::cerr << "MQTT 连接失败: " << e.what() << std::endl;
        return -1;
    }
}

void IpcMqttClient::disconnect()
{
    try {
        if (m_pClient->is_connected()) {
            m_pClient->disconnect()->wait();
        }
    } catch (...) {}
}

json IpcMqttClient::sendCommand(const std::string &strCommand,
                                const json &jData,
                                int nTimeoutMs)
{
    std::string strRequestId = generateRequestId();

```

```

    json jRequest;
    jRequest["Command"] = strCommand;
    jRequest["RequestId"] = strRequestId;
    jRequest["Data"] = jsonData.empty() ? json::object() : jsonData;

    // 发布命令
    auto msg = mqtt::make_message(m_strCommandTopic, jRequest.dump());
    msg->set_qos(1);
    m_pClient->publish(msg)->wait_for(std::chrono::seconds(5));

    // 同步等待响应
    std::unique_lock<std::mutex> lock(m_mtxResponse);
    bool bReady = m_cvResponse.wait_for(lock,
        std::chrono::milliseconds(nTimeoutMs),
        [this, &strRequestId] {
            return m_mapPendingResponses.find(strRequestId) !=
m_mapPendingResponses.end();
        });

    if (!bReady) {
        json jTimeout;
        jTimeout["Return"] = -1;
        jTimeout["error"] = "响应超时";
        return jTimeout;
    }

    json jResponse = m_mapPendingResponses[strRequestId];
    m_mapPendingResponses.erase(strRequestId);
    return jResponse;
}

void IpcMqttClient::setResponseCallback(ResponseCallback cb)
{
    m_fnResponseCallback = std::move(cb);
}

void IpcMqttClient::setEventCallback(EventCallback cb)
{
    m_fnEventCallback = std::move(cb);
}

void IpcMqttClient::onMessage(const std::string &strTopic, const
std::string &strPayload)
{
    try {
        json jPayload = json::parse(strPayload);
        std::string strRequestId = jPayload.value("RequestId", "");

        if (strTopic == m_strResponseTopic) {
            // 同步等待模式：存入 map 并通知

```

```

        {
            std::lock_guard<std::mutex> lock(m_mtxResponse);
            m_mapPendingResponses[strRequestId] = jPayload;
        }
        m_cvResponse.notify_all();

        // 异步回调
        if (m_fnResponseCallback) {
            m_fnResponseCallback(strRequestId, jPayload);
        }
    } else if (strTopic == m_strEventTopic) {
        if (m_fnEventCallback) {
            m_fnEventCallback(jPayload);
        }
    }
} catch (const std::exception &e) {
    std::cerr << "消息解析失败: " << e.what() << std::endl;
}
}

std::string IpcMqttClient::generateRequestId()
{
    static std::random_device rd;
    static std::mt19937 gen(rd());
    std::uniform_int_distribution<uint64_t> dis;

    std::ostringstream oss;
    oss << "req-" << std::hex << dis(gen);
    return oss.str();
}

```

7.3 使用示例

```

#include "IpcMqttClient.h"
#include <iostream>

int main()
{
    // 创建客户端（连接 Broker，订阅响应和事件 Topic）
    IpcMqttClient client("183.129.224.253", 1883, "platform_user",
        "platform_password", "IPC20260001");

    // 设置事件回调（可选，用于接收设备主动上报的告警等）
    client.setEventCallback([](const json &jEvent) {
        std::cout << "收到事件: " << jEvent.dump(2) << std::endl;
    });

    if (client.connect() != 0) {
        std::cerr << "连接失败" << std::endl;
        return 1;
    }
}

```



```
// 发送命令（同步等待响应，10 秒超时）
json jResponse = client.sendCommand("NET_TV_GET_TARGET_LIB");
std::cout << "响应: " << jResponse.dump(2) << std::endl;

return 0;
}
```

8. 命令详细示例

以下所有示例均基于上述 `IpcMqttClient` 封装，展示第三方平台如何向设备发送命令并接收响应。

8.1 添加人脸（NET_TV_ADD_FACE_INFO）

向指定目标库添加一条人脸记录。图片需先通过 FTP/SCP 等方式上传至设备可访问的路径。

请求 Data:

字段	类型	必填	说明
<code>LibId</code>	string	是	目标库名称，需已存在
<code>Name</code>	string	是	人员姓名
<code>PhoneNum</code>	string	否	联系方式
<code>PicPath</code>	string	是	人脸图片在设备上的完整路径
<code>BinPath</code>	string	否	特征值文件路径（留空即可）
<code>PicType</code>	string	是	图片格式，如 <code>jpg</code> 、 <code>png</code>
<code>PicSize</code>	int	是	图片文件大小（字节）
<code>PicDate</code>	string	否	图片拍摄/上传时间
<code>Picwidth</code>	int	否	图片宽度（像素）
<code>PicHeight</code>	int	否	图片高度（像素）

C++ 完整示例:

```
json jData = {
    {"LibId",      "员工库"},
    {"Name",       "张三"},
    {"PhoneNum",   "13800000000"},
    {"PicPath",    "/opt/cam/face/zhangsan.jpg"},
    {"BinPath",    ""},
    {"PicType",    "jpg"},
    {"PicSize",    102400},
    {"PicDate",    "2026-05-22 10:00:00"},
}
```

```

        {"Picwidth", 640},
        {"PicHeight", 480}
    };

    json jResponse = client.sendCommand("NET_TV_ADD_FACE_INFO", jData);
    if (jResponse["Return"] == 0) {
        std::cout << "添加人脸成功" << std::endl;
    } else {
        std::cout << "添加人脸失败, Return=" << jResponse["Return"] <<
        std::endl;
    }
}

```

响应示例:

```

{
    "Command": "NET_TV_ADD_FACE_INFO",
    "RequestId": "req-a1b2c3d4",
    "Return": 0,
    "Data": {}
}

```

8.2 删除人脸 (NET_TV_DEL_FACE_INFO)

根据人脸 ID 列表批量删除人脸记录。ID 来自添加人脸的响应或获取人脸的查询结果。

请求 Data:

字段	类型	必填	说明
Ids	int[]	是	人脸 ID 数组, 最多 128 个

C++ 完整示例:

```

json jData = {
    {"Ids", {10001, 10002, 10003}}
};

json jResponse = client.sendCommand("NET_TV_DEL_FACE_INFO", jData);
if (jResponse["Return"] == 0) {
    std::cout << "删除人脸成功" << std::endl;
} else {
    std::cout << "删除人脸失败, Return=" << jResponse["Return"] <<
    std::endl;
}
}

```

响应示例:

```
{
    "Command": "NET_TV_DEL_FACE_INFO",
    "RequestId": "req-e5f6g7h8",
    "Return": 0,
    "Data": {}
}
```

8.3 修改人脸 (NET_TV_SET_FACE_INFO)

修改已有人员的人脸信息。`Id` 字段为必填，标识要修改的记录。

请求 Data:

字段	类型	必填	说明
<code>Id</code>	int	是	人脸记录 ID (添加时由系统分配)
<code>LibId</code>	string	否	目标库名称
<code>Name</code>	string	否	人员姓名
<code>PhoneNum</code>	string	否	联系方式
<code>PicPath</code>	string	否	新的人脸图片路径
<code>PicType</code>	string	否	图片格式
<code>PicSize</code>	int	否	图片大小 (字节)
<code>PicDate</code>	string	否	图片时间

C++ 完整示例:

```
json jData = {
    {"Id", 10001},
    {"LibId", "员工库"},
    {"Name", "张三-修改"},
    {"PhoneNum", "13900000000"},
    {"PicPath", "/opt/cam/face/zhangsan_new.jpg"},
    {"PicType", "jpg"},
    {"PicSize", 120000},
    {"PicDate", "2026-05-22 11:00:00"}
};

json jResponse = client.sendCommand("NET_TV_SET_FACE_INFO", jData);
if (jResponse["Return"] == 0) {
    std::cout << "修改人脸成功" << std::endl;
} else {
    std::cout << "修改人脸失败, Return=" << jResponse["Return"] <<
std::endl;
}
```

响应示例:

```
{
  "Command": "NET_TV_SET_FACE_INFO",
  "RequestId": "req-i9j0k1l2",
  "Return": 0,
  "Data": {}
}
```

8.4 获取人脸 (NET_TV_GET_FACE_INFO)

查询人脸记录列表。支持按目标库、姓名、联系方式、模型状态、质量等级筛选。传 -1 或空字符串表示不过滤。

请求 Data:

字段	类型	必填	说明
LibId	string	否	目标库名称, 空字符串表示所有库
Name	string	否	按姓名模糊匹配, 空字符串表示不过滤
PhoneNum	string	否	按联系方式过滤, 空字符串表示不过滤
ModelState	int	否	模型状态: -1 全部, 0 未处理, 1 成功, -2 失败
RatingLevel	int	否	质量等级: -1 全部, 0 全部, 1 未知, 2 低, 3 高

C++ 完整示例:

```
// 查询"员工库"中的所有人脸
json jData = {
  {"LibId",      "员工库"},
  {"Name",      ""},
  {"PhoneNum",   ""},
  {"ModelState", -1},
  {"RatingLevel", -1}
};

json jResponse = client.sendCommand("NET_TV_GET_FACE_INFO", jData);
if (jResponse["Return"] == 0) {
  const json &jData = jResponse["Data"];
  int nCount = jData.value("FaceInfoCount", 0);
  std::cout << "人脸数量: " << nCount << std::endl;

  for (const auto &face : jData["FaceInfos"]) {
    std::cout << "  ID=" << face["Id"]
              << " Name=" << face["Name"]
              << " Lib=" << face["FaceLibName"]

```

```
        << " Model=" << face["ModelState"]
        << std::endl;
    }
}
```

响应示例:

```
{
  "Command": "NET_TV_GET_FACE_INFO",
  "RequestId": "req-m3n4o5p6",
  "Return": 0,
  "Data": {
    "FaceInfoCount": 2,
    "FaceInfos": [
      {
        "Id": 10001,
        "FaceLibName": "员工库",
        "Name": "张三",
        "PhoneNum": "13800000000",
        "PicPath": "/opt/cam/face/zhangsan.jpg",
        "BinPath": "",
        "PicType": "jpg",
        "PicSize": 102400,
        "PicDate": "2026-05-22 10:00:00",
        "ModelState": 1,
        "RatingLevel": 3
      },
      {
        "Id": 10002,
        "FaceLibName": "员工库",
        "Name": "李四",
        "PhoneNum": "13900000000",
        "PicPath": "/opt/cam/face/lisi.jpg",
        "BinPath": "",
        "PicType": "jpg",
        "PicSize": 98000,
        "PicDate": "2026-05-22 10:30:00",
        "ModelState": 1,
        "RatingLevel": 2
      }
    ]
  }
}
```

8.5 添加目标库 (NET_TV_ADD_TARGET_LIB)

创建一个新的人脸目标库。库名不可重复。

请求 Data:

字段	类型	必填	说明
LibId	string	是	目标库名称，不可与已有库重名

C++ 完整示例：

```
json jData = {
    {"LibId", "访客库"}
};

json jResponse = client.sendCommand("NET_TV_ADD_TARGET_LIB", jData);
if (jResponse["Return"] == 0) {
    std::cout << "添加目标库成功" << std::endl;
} else {
    std::cout << "添加目标库失败，Return=" << jResponse["Return"] <<
    std::endl;
}
```

响应示例：

```
{
  "Command": "NET_TV_ADD_TARGET_LIB",
  "RequestId": "req-q7r8s9t0",
  "Return": 0,
  "Data": {}
}
```

8.6 删除目标库 (NET_TV_DEL_TARGET_LIB)

删除指定目标库。库下所有人脸记录将一并删除。

请求 Data：

字段	类型	必填	说明
LibId	string	是	要删除的目标库名称

C++ 完整示例：

```

json jData = {
    {"LibId", "访客库"}
};

json jResponse = client.sendCommand("NET_TV_DEL_TARGET_LIB", jData);
if (jResponse["Return"] == 0) {
    std::cout << "删除目标库成功" << std::endl;
} else {
    std::cout << "删除目标库失败, Return=" << jResponse["Return"] <<
std::endl;
}

```

响应示例:

```

{
    "Command": "NET_TV_DEL_TARGET_LIB",
    "RequestId": "req-u1v2w3x4",
    "Return": 0,
    "Data": {}
}

```

8.7 修改目标库 (NET_TV_SET_TARGET_LIB)

重命名已有目标库。需同时提供旧名称和新名称。

请求 Data:

字段	类型	必填	说明
LibId_old	string	是	原目标库名称
LibId_new	string	是	新目标库名称

C++ 完整示例:

```

json jData = {
    {"LibId_old", "员工库"},
    {"LibId_new", "员工库-正式"}
};

json jResponse = client.sendCommand("NET_TV_SET_TARGET_LIB", jData);
if (jResponse["Return"] == 0) {
    std::cout << "修改目标库成功" << std::endl;
} else {
    std::cout << "修改目标库失败, Return=" << jResponse["Return"] <<
std::endl;
}

```

响应示例:

```
{
  "Command": "NET_TV_SET_TARGET_LIB",
  "RequestId": "req-y5z6a7b8",
  "Return": 0,
  "Data": {}
}
```

8.8 获取目标库 (NET_TV_GET_TARGET_LIB)

获取设备上所有目标库列表及其人脸统计信息。Data 传空对象即可。

C++ 完整示例：

```
json jResponse = client.sendCommand("NET_TV_GET_TARGET_LIB");
if (jResponse["Return"] == 0) {
    const json &jData = jResponse["Data"];
    int nCount = jData.value("TargetLibCount", 0);
    std::cout << "目标库数量：" << nCount << std::endl;

    for (const auto &lib : jData["TargetLibInfos"]) {
        std::cout << "  库名=" << lib["FaceLibName"]
                  << "  总数=" << lib["TotalFace"]
                  << "  正常=" << lib["NormalNum"]
                  << "  异常=" << lib["AbnormalNum"]
                  << std::endl;
    }
}
```

响应示例：

```
{
  "Command": "NET_TV_GET_TARGET_LIB",
  "RequestId": "req-c9d0e1f2",
  "Return": 0,
  "Data": {
    "TargetLibCount": 2,
    "TargetLibInfos": [
      {
        "FaceLibName": "员工库",
        "TotalFace": 15,
        "NormalNum": 12,
        "AbnormalNum": 3
      },
      {
        "FaceLibName": "访客库",
        "TotalFace": 8,
        "NormalNum": 8,
        "AbnormalNum": 0
      }
    ]
  }
}
```



```
    ]  
  }  
}
```

8.9 获取垃圾暴露识别配置 (NET_TV_GET_GARBAGE_EXPOSURE_CFG)

查询垃圾暴露智能检测功能的配置，包括启用状态、检测灵敏度、检测区域、布防时间和联动方式。

C++ 完整示例：

```
json jResponse = client.sendCommand("NET_TV_GET_GARBAGE_EXPOSURE_CFG");  
if (jResponse["Return"] == 0) {  
    const json &jCfg = jResponse["Data"];  
    std::cout << "启用： " << jCfg["Enable"] << std::endl;  
    std::cout << "灵敏度： " << jCfg["Rule"]["Sensitivity"] << std::endl;  
    std::cout << "区域顶点数： " << jCfg["Rule"]["Region"]["PointNum"] <<  
    std::endl;  
  
    // 遍历检测区域坐标  
    for (const auto &pt : jCfg["Rule"]["Region"]["Points"]) {  
        std::cout << "  顶点： (" << pt["X"] << ", " << pt["Y"] << ")" <<  
        std::endl;  
    }  
}
```

响应示例：

```
{  
  "Command": "NET_TV_GET_GARBAGE_EXPOSURE_CFG",  
  "RequestId": "req-g3h4i5j6",  
  "Return": 0,  
  "Data": {  
    "Enable": true,  
    "Rule": {  
      "Sensitivity": 60,  
      "Region": {  
        "PointNum": 4,  
        "Points": [  
          {"X": 0.1, "Y": 0.1},  
          {"X": 0.9, "Y": 0.1},  
          {"X": 0.9, "Y": 0.9},  
          {"X": 0.1, "Y": 0.9}  
        ]  
      }  
    },  
    "AlarmTime1": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,  
"MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],  
  }
```

```

        "AlarmTime2": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
"MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
        "AlarmTime3": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
"MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
        "AlarmTime4": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
"MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
        "AlarmTime5": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
"MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
        "AlarmTime6": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
"MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
        "AlarmTime7": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
"MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
        "LinkageMode": {
            "Tradition": [],
            "AlarmLinkage": [],
            "RecordChn": []
        }
    }
}

```

响应字段	类型	说明
<code>Enable</code>	bool	是否启用垃圾暴露检测
<code>Rule.Sensitivity</code>	int	灵敏度 [1-100]，值越大越灵敏
<code>Rule.Region.PointNum</code>	int	检测区域顶点数（最多 32 个）
<code>Rule.Region.Points</code>	array	检测区域顶点数组，每个元素包含 <code>x</code> 、 <code>y</code> （float，归一化坐标 [0.0-1.0]）
<code>AlarmTime1 ~ AlarmTime7</code>	array	布防时间，周一到周日，每天的时间段数组
<code>LinkageMode</code>	object	联动配置（ <code>Tradition</code> 传统联动、 <code>AlarmLinkage</code> 报警输出、 <code>RecordChn</code> 录像通道）

8.10 设置垃圾暴露识别配置 (NET_TV_SET_GARBAGE_EXPOSURE_CFG)

修改垃圾暴露智能检测配置。只需传入要修改的字段，未传入的字段保持不变。

C++ 完整示例（启用检测并设置灵敏度和区域）：

```
json jData = {
    {"Enable", true},
    {"Rule", {
        {"Sensitivity", 80},
        {"Region", {
            {"PointNum", 4},
            {"Points", {
                {"X", 0.05f}, {"Y", 0.05f}},
                {"X", 0.95f}, {"Y", 0.05f}},
                {"X", 0.95f}, {"Y", 0.95f}},
                {"X", 0.05f}, {"Y", 0.95f}}
            }}
        }},
    {"AlarmTime1", {{{{"StartTime", {{{"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}}},
        {"StopTime", {{{"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}}},
    {"AlarmTime2", {{{{"StartTime", {{{"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}}},
        {"StopTime", {{{"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}}},
    {"AlarmTime3", {{{{"StartTime", {{{"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}}},
        {"StopTime", {{{"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}}},
    {"AlarmTime4", {{{{"StartTime", {{{"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}}},
        {"StopTime", {{{"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}}},
    {"AlarmTime5", {{{{"StartTime", {{{"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}}},
        {"StopTime", {{{"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}}},
    {"AlarmTime6", {{{{"StartTime", {{{"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}}},
        {"StopTime", {{{"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}}},
    {"AlarmTime7", {{{{"StartTime", {{{"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}}},
        {"StopTime", {{{"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}}},
    {"LinkageMode", {
        {"Tradition", {0}},
    }
```

```

        {"AlarmLinkage", {0}},
        {"RecordChn", {0}}
    }}
};

json jResponse = client.sendCommand("NET_TV_SET_GARBAGE_EXPOSURE_CFG",
jData);
if (jResponse["Return"] == 0) {
    std::cout << "设置垃圾暴露配置成功" << std::endl;
}

```

C++ 完整示例（仅关闭检测）：

```

json jData = {
    {"Enable", false}
};

json jResponse = client.sendCommand("NET_TV_SET_GARBAGE_EXPOSURE_CFG",
jData);

```

响应示例：

```

{
    "Command": "NET_TV_SET_GARBAGE_EXPOSURE_CFG",
    "RequestId": "req-k7l8m9n0",
    "Return": 0,
    "Data": {}
}

```

8.11 获取垃圾满溢识别配置 (NET_TV_GET_GARBAGE_OVERFLOW_CFG)

查询垃圾满溢智能检测功能的配置。与垃圾暴露配置结构相同。

C++ 完整示例：

```

json jResponse = client.sendCommand("NET_TV_GET_GARBAGE_OVERFLOW_CFG");
if (jResponse["Return"] == 0) {
    const json &jCfg = jResponse["Data"];
    std::cout << "启用： " << jCfg["Enable"] << std::endl;
    std::cout << "灵敏度： " << jCfg["Rule"]["Sensitivity"] << std::endl;

    for (const auto &pt : jCfg["Rule"]["Region"]["Points"]) {
        std::cout << " 顶点： (" << pt["x"] << ", " << pt["y"] << ")" <<
std::endl;
    }
}

```

响应示例：

```
{
  "Command": "NET_TV_GET_GARBAGE_OVERFLOW_CFG",
  "RequestId": "req-o1p2q3r4",
  "Return": 0,
  "Data": {
    "Enable": true,
    "Rule": {
      "Sensitivity": 50,
      "Region": {
        "PointNum": 4,
        "Points": [
          {"X": 0.2, "Y": 0.2},
          {"X": 0.8, "Y": 0.2},
          {"X": 0.8, "Y": 0.8},
          {"X": 0.2, "Y": 0.8}
        ]
      }
    },
    "AlarmTime1": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
    "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
    "AlarmTime2": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
    "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
    "AlarmTime3": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
    "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
    "AlarmTime4": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
    "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
    "AlarmTime5": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
    "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
    "AlarmTime6": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
    "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
    "AlarmTime7": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0,
    "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
    "LinkageMode": {
      "Tradition": [],
      "AlarmLinkage": [],
      "RecordChn": []
    }
  }
}
```

响应字段	类型	说明
<code>Enable</code>	bool	是否启用垃圾满溢检测
<code>Rule.Sensitivity</code>	int	灵敏度 [1-100]
<code>Rule.Region.PointNum</code>	int	检测区域顶点数
<code>Rule.Region.Points</code>	array	检测区域顶点数组，每个元素包含 <code>x</code> 、 <code>y</code>
<code>AlarmTime1</code> ~ <code>AlarmTime7</code>	array	布防时间，周一到周日
<code>LinkageMode</code>	object	联动配置

8.12 设置垃圾满溢识别配置 (NET_TV_SET_GARBAGE_OVERFLOW_CFG)

修改垃圾满溢智能检测配置。与垃圾暴露配置结构相同。

C++ 完整示例（启用检测，设置灵敏度和区域）：

```
json jData = {
    {"Enable", true},
    {"Rule", {
        {"Sensitivity", 70},
        {"Region", {
            {"PointNum", 4},
            {"Points", {
                {"X", 0.1f}, {"Y", 0.1f}},
                {"X", 0.9f}, {"Y", 0.1f}},
                {"X", 0.9f}, {"Y", 0.9f}},
                {"X", 0.1f}, {"Y", 0.9f}}
            }}
        }},
    {"AlarmTime1", {{{"StartTime", {"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}},
        {"StopTime", {"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}},
    {"AlarmTime2", {{{"StartTime", {"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}},
        {"StopTime", {"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}},
    {"AlarmTime3", {{{"StartTime", {"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}},
        {"StopTime", {"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}},
    {"AlarmTime4", {{{"StartTime", {"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}},
        {"StopTime", {"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}},
}
```

```

        {"AlarmTime5", {{{"StartTime", {"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}},
                                {"StopTime", {"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}},
        {"AlarmTime6", {{{"StartTime", {"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}},
                                {"StopTime", {"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}},
        {"AlarmTime7", {{{"StartTime", {"Hour", 0}, {"Min", 0}, {"Sec", 0}, {"MSec", 0}}},
                                {"StopTime", {"Hour", 23}, {"Min", 59}, {"Sec", 59}, {"MSec", 0}}}}},
        {"LinkageMode", {
            {"Tradition", {0}},
            {"AlarmLinkage", {0}},
            {"RecordChn", {0}}
        }}
    };

    json jResponse = client.sendCommand("NET_TV_SET_GARBAGE_OVERFLOW_CFG",
    jData);
    if (jResponse["Return"] == 0) {
        std::cout << "设置垃圾满溢配置成功" << std::endl;
    }
}

```

C++ 完整示例（仅关闭检测）：

```

json jData = {
    {"Enable", false}
};

json jResponse = client.sendCommand("NET_TV_SET_GARBAGE_OVERFLOW_CFG",
jData);

```

响应示例：

```

{
    "Command": "NET_TV_SET_GARBAGE_OVERFLOW_CFG",
    "RequestId": "req-s5t6u7v8",
    "Return": 0,
    "Data": {}
}

```

9. 布防时间说明

布防时间使用 AlarmTime1 ~ AlarmTime7 字段，分别对应周一到周日。每个字段是时间段数组，每天最多 8 个时间段。

每个时间段包含 StartTime 和 StopTime，各自包含：

字段	类型	说明
Hour	int	小时 [0-23]
Min	int	分钟 [0-59]
Sec	int	秒 [0-59]
MSec	int	毫秒 [0-999]

全天布防示例：

```
{
  "AlarmTime1": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0, "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
  "AlarmTime2": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0, "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
  "AlarmTime3": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0, "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
  "AlarmTime4": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0, "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
  "AlarmTime5": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0, "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
  "AlarmTime6": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0, "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}],
  "AlarmTime7": [{"StartTime": {"Hour": 0, "Min": 0, "Sec": 0, "MSec": 0}, "StopTime": {"Hour": 23, "Min": 59, "Sec": 59, "MSec": 0}}]
}
```

仅工作日白天示例：

```
{
  "AlarmTime1": [],
  "AlarmTime2": [{"StartTime": {"Hour": 8, "Min": 0, "Sec": 0, "MSec": 0}, "StopTime": {"Hour": 18, "Min": 0, "Sec": 0, "MSec": 0}}],
  "AlarmTime3": [{"StartTime": {"Hour": 8, "Min": 0, "Sec": 0, "MSec": 0}, "StopTime": {"Hour": 18, "Min": 0, "Sec": 0, "MSec": 0}}],
  "AlarmTime4": [{"StartTime": {"Hour": 8, "Min": 0, "Sec": 0, "MSec": 0}, "StopTime": {"Hour": 18, "Min": 0, "Sec": 0, "MSec": 0}}],
  "AlarmTime5": [{"StartTime": {"Hour": 8, "Min": 0, "Sec": 0, "MSec": 0}, "StopTime": {"Hour": 18, "Min": 0, "Sec": 0, "MSec": 0}}],
  "AlarmTime6": [{"StartTime": {"Hour": 8, "Min": 0, "Sec": 0, "MSec": 0}, "StopTime": {"Hour": 18, "Min": 0, "Sec": 0, "MSec": 0}}],
  "AlarmTime7": []
}
```

如果某天不需要布防，对应字段传空数组 [] 或不传该字段。

10. 检测区域坐标说明

检测区域使用 `Region` 对象，包含 `PointNum`（顶点数）和 `Points`（顶点数组）。每个顶点包含 `x`、`y` 归一化坐标（float 类型）。



- X 坐标: 0.0 = 画面最左, 1.0 = 画面最右
- Y 坐标: 0.0 = 画面最上, 1.0 = 画面最下
- 至少需要 3 个顶点构成检测区域, 最多 32 个顶点
- 顶点按顺时针或逆时针顺序排列

JSON 格式:

```
{
  "Region": {
    "PointNum": 4,
    "Points": [
      {"X": 0.1, "Y": 0.1},
      {"X": 0.9, "Y": 0.1},
      {"X": 0.9, "Y": 0.9},
      {"X": 0.1, "Y": 0.9}
    ]
  }
}
```